

算法分析第2次作业

算法分析题2

2-3

2-3 设 $a[0:n-1]$ 是已排好序的数组。请改写二分搜索算法，使得当搜索元素 x 不在数组中时，返回小于 x 的最大元素位置 i 和大于 x 的最小元素位置 j 。当搜索元素在数组中时， i 和 j 相同，均为 x 在数组中的位置。

第2-3题答案:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  template<class T>
5  requires totally_ordered<T>
6  void binary_search(T a[], const T& x, size_t left, size_t right, size_t& i,
7  size_t& j) {
8      size_t middle;
9      while(left ≤ right){
10         middle = (left + right) / 2;
11         if (x == a[middle]) {
12             i = middle, j = middle;
13             return;
14         }
15         if (x > a[middle]) {
16             left = middle + 1;
17         }
18         else {
19             right = middle - 1;
20         }
21     }
22     i = right;
23     j = left;
24 }
25
26 int main() {
```

```

26     int a[] = { 2,3,4,5,6,7,8,9,11,12,13,14,15,16,17,18 };
27     cout << "The list a is ";
28     for (int i = 0; i < 16; i++) cout << a[i] << ' ';
29     cout << '\n';
30     size_t i, j;
31     binary_search(a, 8, (size_t)0, (size_t)15, i, j);
32     cout << "The found i = " << i << ", j = " << j << "\n";
33     binary_search(a, 10, (size_t)0, (size_t)15, i, j);
34     cout << "The found i = " << i << ", j = " << j << "\n";
35     return 0;
36 }

```

2-4

2-4 给定两个大整数 u 和 v ，它们分别有 m 和 n 位数字，且 $m \leq n$ 。用通常的乘法求 uv 的值需要 $O(mn)$ 时间。可以将 u 和 v 均看作有 n 位数字的大整数，用本章介绍的分治法，在 $O(n^{\log_3})$ 时间内计算 uv 的值。当 m 比 n 小得多时，用这种方法就显得效率不够高。试设计一个算法，在上述情况下用 $O(nm^{\log_2 3})$ 时间求出 uv 的值。

第2-4题答案:

设两个大整数 u 和 v ，位数分别为 m 和 n ，且满足 $m \ll n$

被乘数 u 保持不变

设 $k = \lceil n/m \rceil$ 位总段数，那么乘数 v 可以表示为：

$$v = \sum_{i=0}^{k-1} v_i (10^m)^i$$

其中 $0 \leq v_i < 10^m$

因此根据分配律可知：

$$uv = u \cdot \left(\sum_{i=0}^{k-1} v_i (10^m)^i \right) = \sum_{i=0}^{k-1} ((u \cdot v_i) \cdot 10^{mi})$$

所以我们对于每一个局部乘积 $u \cdot v_i$ 可以使用 *Karatsuba* 分治法

因此单次运算的时间复杂度为：

$$T_{local} = O(m^{\log_2 3})$$

而 $k \approx n/m$

因此总的复杂度为：

$$\begin{aligned}
 T &= \frac{n}{m} \cdot O(m^{\log_2 3}) \\
 &= O\left(\frac{n}{m} \cdot m^{\log_2 3}\right) \\
 &= O(n \cdot m^{\log_2 3 - 1}) \\
 &= O(nm^{\log_2 \frac{3}{2}})
 \end{aligned}$$

2-5

2-5 在用分治法求两个 n 位大整数 u 和 v 的乘积时, 将 u 和 v 都分割为长度为 $n/3$ 位的 3 段。证明可以用 5 次 $n/3$ 位整数的乘法求得 uv 的值。按此思想设计一个求两个大整数乘积的分治算法, 并分析算法的计算复杂性 (提示: n 位的大整数除以一个常数 k 可以在 $\theta(n)$ 时间内完成。符号 θ 所隐含的常数可能依赖于 k)。

第2-5题答案:

首先, 我们将 n 位的大整数 u 和 v 按照 $m = n/3$ 位进行切割。

设基数 $x = 2^{n/3}$, 则:

$$u = u_2x^2 + u_1x + u_0$$

$$v = v_2x^2 + v_1x + v_0$$

这里的 u_i, v_i 都是小于 $2^{n/3}$ 的整数。我们要计算的是:

$$w(x) = u(x) \cdot v(x) = w_4x^4 + w_3x^3 + w_2x^2 + w_1x + w_0$$

为了求出这 5 个未知系数 w_0, w_1, w_2, w_3, w_4 , 我们需要 5 个独立的方程。我们选取 x 的 5 个简单值: $0, 1, -1, 2, -2$ 。

计算每个 x_i 对应的 $u(x_i)$ 和 $v(x_i)$:

1. $x = 0$: $U_0 = u_0, \quad V_0 = v_0$

2. $x = 1$: $U_1 = u_2 + u_1 + u_0, \quad V_1 = v_2 + v_1 + v_0$

3. $x = -1$: $U_{-1} = u_2 - u_1 + u_0, \quad V_{-1} = v_2 - v_1 + v_0$

4. $x = 2$: $U_2 = 4u_2 + 2u_1 + u_0, \quad V_2 = 4v_2 + 2v_1 + v_0$

5. $x = -2$: $U_{-2} = 4u_2 - 2u_1 + u_0, \quad V_{-2} = 4v_2 - 2v_1 + v_0$

然后我们执行 5 次规模为 $n/3$ 位的乘法, 得到 5 个点值:

- $a = w(0) = U_0 \cdot V_0$
- $e = w(1) = U_1 \cdot V_1$
- $d = w(-1) = U_{-1} \cdot V_{-1}$
- $c = w(2) = U_2 \cdot V_2$
- $b = w(-2) = U_{-2} \cdot V_{-2}$

现在我们有 5 个方程, 构成一个范德蒙德矩阵系统:

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 & 1 \\ 1 & 2 & 4 & 8 & 16 \\ 1 & -2 & 4 & -8 & 16 \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ w_3 \\ w_4 \end{bmatrix} = \begin{bmatrix} a \\ e \\ d \\ c \\ b \end{bmatrix}$$

我们要解的是一个典型的范德蒙德型线性方程组。为了让计算过程清晰，我们首先写出对应的增广矩阵，并利用其结构的**对称性**（奇数项和偶数项的抵消）来简化消元过程。

1. 初始增广矩阵

$$\left[\begin{array}{ccccc|c} 1 & 0 & 0 & 0 & 0 & a \\ 1 & 1 & 1 & 1 & 1 & e \\ 1 & -1 & 1 & -1 & 1 & d \\ 1 & 2 & 4 & 8 & 16 & c \\ 1 & -2 & 4 & -8 & 16 & b \end{array} \right]$$

从第一行直接得到：

$$w_0 = a$$

我们将 $w_0 = a$ 代入其余四行，并将这四行两两组合，从而将变量 w_1, w_3 与 w_2, w_4 分离开来。

定义变量代换：

- $L_1 : w_1 + w_2 + w_3 + w_4 = e - a$
- $L_2 : -w_1 + w_2 - w_3 + w_4 = d - a$
- $L_3 : 2w_1 + 4w_2 + 8w_3 + 16w_4 = c - a$
- $L_4 : -2w_1 + 4w_2 - 8w_3 + 16w_4 = b - a$

第一组：求 w_2 和 w_4 (相加)

- $(L_1 + L_2)/2 \implies w_2 + w_4 = \frac{d+e-2a}{2}$ —— (方程 A)
- $(L_3 + L_4)/2 \implies 4w_2 + 16w_4 = \frac{b+c-2a}{2}$ —— (方程 B)

第二组：求 w_1 和 w_3 (相减)

- $(L_1 - L_2)/2 \implies w_1 + w_3 = \frac{e-d}{2}$ —— (方程 C)
- $(L_3 - L_4)/2 \implies 2w_1 + 8w_3 = \frac{c-b}{2}$ —— (方程 D)

利用方程 A 和 B：

1. 由 A 得： $4w_2 + 4w_4 = 2(d + e - 2a)$

2. 用 B 减去上式：

$$(16 - 4)w_4 = \frac{b+c-2a}{2} - 2(d + e - 2a)$$

$$12w_4 = \frac{b+c-2a-4d-4e+8a}{2} = \frac{b+c-4(d+e)+6a}{2}$$

$$w_4 = \frac{b+c-4(d+e)+6a}{24}$$

3. 代回 A 求解 w_2 :

$$w_2 = \frac{d+e-2a}{2} - w_4 = \frac{12(d+e-2a)-(b+c-4d-4e+6a)}{24}$$

$$w_2 = \frac{12d+12e-24a-b-c+4d+4e-6a}{24}$$

$$w_2 = \frac{-b-c+16(d+e)-30a}{24}$$

利用方程 C 和 D:

1. 由 C 得: $2w_1 + 2w_3 = e - d$

2. 用 D 减去上式:

$$(8-2)w_3 = \frac{c-b}{2} - (e-d) = \frac{c-b-2e+2d}{2}$$

$$6w_3 = \frac{-b+c+2(d-e)}{2}$$

$$w_3 = \frac{-b+c+2(d-e)}{12}$$

3. 代回 C 求解 w_1 :

$$w_1 = \frac{e-d}{2} - w_3 = \frac{6(e-d)-(-b+c+2d-2e)}{12}$$

$$w_1 = \frac{6e-6d+b-c-2d+2e}{12}$$

$$w_1 = \frac{b-c-8(d-e)}{12}$$

综上通过高斯消元, 解出系数:

$$1. w_0 = a$$

$$2. w_1 = \frac{b-c-8(d-e)}{12}$$

$$3. w_2 = \frac{-b-c+16(d+e)-30a}{24}$$

$$4. w_3 = \frac{-b+c+2(d-e)}{12}$$

$$5. w_4 = \frac{b+c-4(d+e)+6a}{24}$$

使用主定理分析递归式:

$$T(n) = 5T(n/3) + O(n)$$

1. 参数提取: $a = 5$ (子问题个数), $b = 3$ (规模缩减倍数), $f(n) = O(n)$ 。

2. 关键值计算: 计算临界幂 $\log_b a = \log_3 5$ 。

- 因为 $3^1 < 5 < 3^2$, 所以 $1 < \log_3 5 < 2$ 。
- 具体数值: $\log_3 5 \approx 1.465$ 。

3. 比较: 比较 $f(n) = n^1$ 与 $n^{\log_3 5}$ 。

- 由于 $1 < 1.465$, 根据主定理第一类情况 (子问题开销占主导):

$$T(n) = \Theta(n^{\log_3 5})$$

结论:

本算法(3路分治): $O(n^{\log_3 5}) \approx O(n^{1.465})$

可以看到, 分治的份数越多, 虽然单层合并变复杂了, 但整体幂次会降低。

2-8

2-8 设 $a[0:n-1]$ 是有 n 个元素的数组, k ($0 \leq k \leq n-1$) 是一个非负整数。试设计一个算法将子数组 $a[0:k-1]$ 与 $a[k:n-1]$ 换位。要求算法在最坏情况下耗时 $O(n)$, 且只用到 $O(1)$ 的辅助空间。

第2-8题答案:

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  template<class T>
5  void reverse(T a[], size_t i, size_t j) {
6      while (i < j) {
7          swap(a[i], a[j]);
8          i++, j--;
9      }
10 }
11
12 template<class T>
13 void solve(T a[], size_t n, size_t k) {
14     reverse(a, 0, k - 1);
15     reverse(a, k, n - 1);
16     reverse(a, 0, n - 1);
17 }
18
19 int main() {
20     int a[] = { 2,3,4,5,6,7,8,9,11,12,13,14,15,16,17,18 };
21     cout << "The list a is ";
22     for (int i = 0; i < 16; i++) cout << a[i] << ' ';
23     cout << '\n';
24     solve(a, (size_t)16, (size_t)7);
25     cout << "The change list a is ";
26     for (int i = 0; i < 16; i++) cout << a[i] << ' ';
27     cout << '\n';
28     return 0;

```

2-9

2-9 设子数组 $a[0:k-1]$ 和 $a[k:n-1]$ 已排好序 ($0 \leq k \leq n-1$)。试设计一个合并这两个子数组为排好序的数组 $a[0:n-1]$ 的算法。要求算法在最坏情况下所用的计算时间为 $O(n)$ ，且只用到 $O(1)$ 的辅助空间。

第2-9题答案:

将数组划分为 $\sqrt{n}$ 个块，利用数组中最大的 $\sqrt{n}$ 个元素作为“内部缓存块”，通过块交换和局部排序实现全局有序。

1. 提取内部缓存块

- 从已排序的两个子数组 $a[0:k-1]$ 和 $a[k:n-1]$ 的末尾各取一部分，通过原地合并收集出整个数组中最大的 $m = \lceil \sqrt{n} \rceil$ 个元素。
- 将这 m 个最大元素集中存放在数组的最右端或最左端，作为“内部缓存区”。
- 复杂度分析**：此步通过局部移动实现，耗时 $O(n)$ 。

2. 块划分与排序

- 将剩余的 $n - m$ 个元素划分为若干个长度为 m 的块。
- 使用**选择排序**的思想，根据每个块的**末尾元素**大小，对这些块进行整体排序。
- 如果末尾元素相同，则比较首元素。
- 复杂度分析**：块的数量约为 $\sqrt{n}$ ，选择排序的复杂度为 $O((\sqrt{n})^2) = O(n)$ 。

3. 识别待合并区间

- 遍历排序后的块，识别出两段待合并的逻辑区间。
- 第一段区间：从当前位置开始，连续的、且块与块之间保持递增关系的块序列。
- 第二段区间：紧随第一段区间后的下一个块。
- 复杂度分析**：只需比较相邻块的边界，总遍历耗时 $O(n)$ 。

4. 利用缓存块进行归并

- 利用步骤1提取出的 m 个最大元素作为缓冲区。
- 由于第二段区间（单个块）的长度不超过缓冲区大小 m ，可以将该块与前一段区间进行类似标准归并的操作。
- 在合并过程中，将较小的元素移到前面，原有的缓冲区元素则被置换到后面。

- **复杂度分析：**每个元素在合并过程中被移动的次数为常数次，总耗时 $O(n)$ 。

5. 缓冲区位移与循环

- 每完成一次区间合并，内部缓存块（因为其元素值最大）自然会“滚动”到已合并部分的右侧。
- 重复步骤 3、4、5，直到处理完所有块。

6. 最终清理

- 最后，数组中只剩下最右端的内部缓存块（即最初提取的那 m 个最大元素）尚未排入最终位置。
- 对这 m 个元素使用基础排序（如插入排序）或将其与前面的元素进行最后一次原地归并。
- **复杂度分析：**对 $\sqrt{n}$ 个元素进行 $O(m^2)$ 排序耗时 $O(n)$ 。

算法实现题2

2-1

第2-1题答案:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  unordered_map<int, int> m;
5  int n, k;
6
7  int main() {
8      freopen("input.txt", "r", stdin);
9      freopen("output.txt", "w", stdout);
10     cin >> n;
11     while (n--) {
12         cin >> k;
13         m[k]++;
14     }
15     int M = 0, s;
16     for (const auto& e : m) {
17         if (e.second > M) {
18             M = e.second;
19             s = e.first;
20         }
21     }
22     cout << s << '\n' << M << '\n';
23     return 0;
```

2-7

第2-7题答案:

观察发现:

$n = 0$ 时只能划分为 $\{\}$ 这一个, 内部用 $\{\}$ 的个数为0

$n = 1$ 时只能划分为 $\{\{1\}\}$ 这一个, 内部用 $\{\}$ 的个数为1

$n = 2$ 时只能划分为 $\{\{1\}, \{2\}\}$ 和 $\{\{1, 2\}\}$ 共2种, 内部用 $\{\}$ 的个数为3

$n = 3$ 时划分为 $\{\{1\}, \{2\}, \{3\}\}$ 、 $\{\{1, 2\}, \{3\}\}$ 、 $\{\{1, 3\}, \{2\}\}$ 、 $\{\{2, 3\}, \{1\}\}$ 、 $\{\{1, 2, 3\}\}$ 共5种, 内部用 $\{\}$ 的个数为10

考虑 $n = k$ 时, 假设此时划分的种类数量为 $f(k)$, 总共的内部元素为 $g(k)$

则若拿走 k , 则剩下的划分的种类的数量为 $f(k - 1)$, 剩下的内部元素为 $g(k - 1)$

我们可以发现每次 n 增加1, 那么就是从空的位置 $f(k - 1)$ 或者内部元素 $g(k - 1)$ 选择一个位置填充 n

因此我们发现 $f(k) = f(k - 1) + g(k - 1)$

那么目标就转换为求出 $g(k)$

通过观察, 我们发现, 我们假设从 $n = k$ 拿出 j 个作为一个特定的子集, 然后再在剩下的 $k - j$ 种进行划分, 因此通过组合数学我们得到

$$g(k) = \sum_{j=1}^k (C_k^j \cdot f(k - j))$$

因此我们得到了两个公式

$$f(k) = f(k - 1) + g(k - 1) \text{ 和 } g(k) = \sum_{j=1}^k (C_k^j \cdot f(k - j))$$

即得到了

$$f(k + 1) = f(k) + \sum_{j=1}^k (C_k^j \cdot f(k - j))$$

因此

$$f(k + 1) = \sum_{j=0}^k (C_k^j \cdot f(k - j))$$

又根据组合数的公式

$$C_n^k = C_{n-1}^{k-1} + C_{n-1}^k$$

即杨辉三角, 我们可以快速的通过打开一个 $(n + 1) \times (n + 1)$ 的表格预计算出所有的结果

代码如下:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  using ll = long long;
4
5  ll n;
6
7  static vector<vector<ll>> get_C(const ll n) {
8      vector<vector<ll>> C(n + 1, vector<ll>(n + 1, 0));
9      for (ll i = 0; i ≤ n; i++) {
10         C[i][0] = 1;
11         for (ll j = 1; j ≤ i; j++) {
12             C[i][j] = C[i - 1][j - 1] + C[i - 1][j];
13         }
14     }
15     return C;
16 }
17
18 static ll solve(const ll n, const vector<vector<ll>>& C) {
19     vector<ll> f(n + 1);
20     f[0] = 1;
21     for (ll i = 0; i < n; i++) {
22         f[i + 1] = 0;
23         for (ll j = 0; j ≤ i; j++) {
24             f[i + 1] += C[i][j] * f[i - j];
25         }
26     }
27     return f[n];
28 }
29
30 int main() {
31     while (cin >> n) {
32         auto C = get_C(n);
33         auto ans = solve(n, C);
34         cout << ans << endl;
35     }
36     return 0;
37 }
```

补充题

(1)存在序关系

```
1  #include <bits/stdc++.h>
2  #include <optional>
3  using namespace std;
4
5  template<class T>
6  requires totally_ordered<T>
7  T quick_select(vector<T>& nums, const int left, const int right, const int k)
8  {
9
10     if (left == right) return nums[left];
11
12     auto pivot = nums[left + (right - left) / 2];
13     auto i = left - 1, j = right + 1;
14     while (i < j) {
15         do i++; while (nums[i] < pivot);
16         do j--; while (nums[j] > pivot);
17         if (i < j) swap(nums[i], nums[j]);
18     }
19
20     auto cnt = j - left + 1;
21     if (k ≤ cnt) return quick_select(nums, left, j, k);
22     else return quick_select(nums, j + 1, right, k - cnt);
23 }
24
25 template<class T>
26 requires totally_ordered<T>
27 optional<T> solve(vector<T>& nums) {
28     int n = (int)nums.size();
29     auto candidate = quick_select(nums, 0, n - 1, (n + 1) / 2);
30
31     auto cnt = 0;
32     for (const auto& e : nums) if (e == candidate) cnt++;
33     if (cnt > n / 2) return candidate;
34     return nullopt;
35 }
36
37 template<class T>
38 ostream& operator<<(ostream& os, const optional<T> opt) {
39     if (opt.has_value()) {
40         os << "Some(" << opt.value() << ")";
41     }
42     else {
```

```

41         os << "None";
42     }
43     return os;
44 }
45
46 struct Point {
47     int x, y;
48 };
49
50 int main() {
51     // 测试用例 1: 整数数组 (有主元素 3)
52     std::vector<int> nums1 = { 3, 1, 3, 2, 3, 3, 4 };
53     // 测试用例 2: 整数数组 (无主元素)
54     std::vector<int> nums2 = { 1, 2, 3, 4 };
55
56     // 测试用例 3: 坐标数组 (有主元素 {1,1})
57     //std::vector<Point> pts1 = { {1,1}, {2,2}, {1,1}, {3,3}, {1,1} };
58     // 测试用例 4: 坐标数组 (无主元素)
59     //std::vector<Point> pts2 = { {0,0}, {1,1}, {2,2} };
60
61     cout << "--- 算法测试开始 ---" << endl;
62
63     cout << "测试用例1: " << solve(nums1) << endl;
64     cout << "测试用例2: " << solve(nums2) << endl;
65
66     //solve(pts1);
67     //solve(pts2);
68
69     return 0;
70 }

```

(2)不存在序关系

```

1  #include <bits/stdc++.h>
2  #include <optional>
3  using namespace std;
4
5  template<class T>
6  requires equality_comparable<T>
7  optional<T> pair_choose(vector<T>& nums) {
8      if (nums.size() == 0)return nullopt;
9      if (nums.size() == 1)return nums[0];
10     int n = (int)nums.size();

```

```

11     vector<T> next_nums;
12     if (n % 2 == 1) {
13         next_nums.push_back(nums[0]);
14         for (int i = 1; i < n; i += 2) {
15             if (nums[i] == nums[i + 1]) next_nums.push_back(nums[i]);
16         }
17     }
18     else {
19         for (int i = 0; i < n; i += 2) {
20             if (nums[i] == nums[i + 1]) next_nums.push_back(nums[i]);
21         }
22     }
23     return pair_choose(next_nums);
24 }
25
26 template<class T>
27 requires equality_comparable<T>
28 optional<T> solve(vector<T>& nums) {
29     int n = (int)nums.size();
30     auto candidate = pair_choose(nums);
31
32     auto cnt = 0;
33     for (const auto& e : nums) if (e == candidate) cnt++;
34     if (cnt > n / 2) return candidate;
35     return nullopt;
36 }
37
38 template <typename T>
39 concept can_stream_out = requires(std::ostream & os, T const& a) {
40     { os << a } → std::convertible_to<std::ostream&>;
41 };
42
43 template<class T>
44 requires can_stream_out<T>
45 ostream& operator<<(ostream& os, const optional<T>& opt) {
46     if (opt.has_value()) {
47         os << "Some(" << opt.value() << ")";
48     }
49     else {
50         os << "None";
51     }
52     return os;
53 }
54

```

```

55 struct Point {
56     int x, y;
57     bool operator==(const Point&) const = default;
58 };
59
60 ostream& operator<<(ostream& os, const Point& pt) {
61     os << "(" << pt.x << ", " << pt.y << ")";
62     return os;
63 }
64
65 int main() {
66     // 测试用例 1: 整数数组 (有主元素 3)
67     std::vector<int> nums1 = { 3, 1, 3, 2, 3, 3, 4 };
68     // 测试用例 2: 整数数组 (无主元素)
69     std::vector<int> nums2 = { 1, 2, 3, 4 };
70
71     // 测试用例 3: 坐标数组 (有主元素 {1,1})
72     std::vector<Point> pts1 = { {1,1}, {2,2}, {1,1}, {3,3}, {1,1} };
73     // 测试用例 4: 坐标数组 (无主元素)
74     std::vector<Point> pts2 = { {0,0}, {1,1}, {2,2} };
75
76     cout << "--- 算法测试开始 ---" << endl;
77
78     cout << "测试用例1: " << solve(nums1) << endl;
79     cout << "测试用例2: " << solve(nums2) << endl;
80
81     cout << "测试用例3: " << solve(pts1) << endl;
82     cout << "测试用例4: " << solve(pts2) << endl;
83
84     return 0;
85 }

```

注

以上代码均使用C++20在Windows下的Visual Studio 2026编译并运行成功